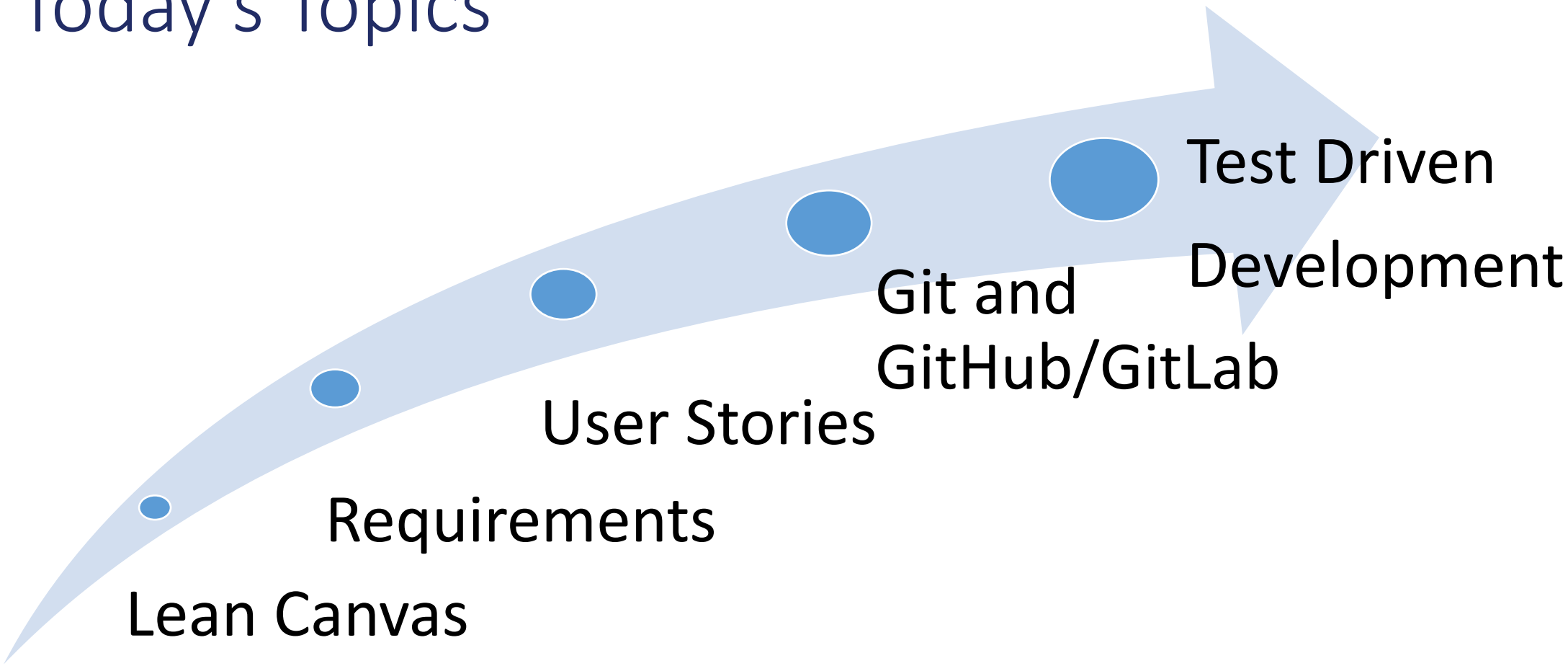
Requirements, Coordination, and Test-Driven Development

FINTECH 512
Software Engineering

Duke

Today's Topics



- Software Engineering: systematic, disciplined, quantifiable
- Building systems, not just code
- Failures come from missing decisions

Lean Canvas

Understanding the “Why” Before the ”What”

What is a Lean Canvas?

One-page business to capture and validate assumptions on a problem worth solving.

Why use a Lean Canvas?

- Forces stakeholders to articulate the core problem and target users
- Identifies assumptions that need validation before building
- Aligns team on business viability alongside technical feasibility
- Prevents building the wrong thing efficiently

The Lean Canvas

Designed for: Startup Name

Designed by: Name1, Name2, …

Date: DD/MM/YYYY

Version: X.Y

Problem

Top 3 problems

Solution

Top 3 features

Unique Value Prop.

Single, clear and compelling message that states why you are different and worth buying

Unfair Advantage

Can't be easily copied or bought

Customer Segments

Target Customers

Existing Alternatives

List how these problems are solved today.

Key Metrics

Key activities you measure

High-Level Concept

List your X for Y analogy

(e.g. YouTube = Flickr for videos)

Channels

Path to customers

Early Adopters

List the characteristics of your ideal customers.

Cost Structure

List your fixed and variable costs.

Customer acquisition costs

Distribution costs

Hosting

People

Etc.

Revenue Streams

List your sources of revenue.

Revenue Model

Life Time Value

Revenue

Gross Margin

Lean Canvas is adapted from The Business Model Canvas (www.businessmodelgeneration.com/canvas). PowerPoint implementation by: Neos Chronos Limited (https://neoschronos.com). License: CC BY-SA 3.0

Example System: BluePay

Extension of Duke's ID Card to allows students and employees of Duke University and the associated healthcare system to use their ID card just like other recent financial systems like PayPal, Stripe, and Venmo.

Key features:

- Web and mobile account access
- Point of sale access
- Social and vendor payments
- Credit available

The Lean Canvas

Designed for: Duke Blue Pay

Designed by: John

Date: 01/12/2025

Version: 1.0

Problem

- Limited functionality of current Duke ID cards.
- No seamless integration with modern payment systems.
- Inconvenience in accessing funds or making payments.

Solution

- Web and mobile account management.
- Point-of-sale access.
- Social and vendor payments.
- Credit availability.

Unique Value Prop.

Transform Duke ID cards into a versatile financial tool

Unfair Advantage

- Exclusive integration with Duke University's ecosystem.
- Leveraging existing Duke ID infrastructure.

Customer Segments

- Duke University students.
- Duke University employees.
- Associated healthcare system staff.

Existing Alternatives

List how these problems are solved today.

Key Metrics

- Number of active users.
- Volume of transactions processed.
- Average transaction size.
- Vendor partnerships established.

High-Level Concept

C omparable to PayPal, Stripe, and Venmo but tailored for the Duke ecosystem.

Channels

- Duke University onboarding communications.
- Email campaigns.
- University and healthcare system portals.
- Campus events.
- influencers

Early Adopters

List the characteristics of your ideal customers.

Cost Structure

- Development and maintenance of the BluePay platform.
- Integration with payment systems.
- Marketing and user support.
- Customer support

Revenue Streams

- Transaction fees from vendors.
- Interest on credit balances.
- Partnership with vendors for exclusive deals.

Lean Canvas is adapted from The Business Model Canvas (www.businessmodelgeneration.com/canvas). PowerPoint implementation by: Neos Chronos Limited (https://neoschronos.com). License: CC BY-SA 3.0

What is a Requirement

- What the system will do, but not how to do it.
- Needed behavior or constraint

Functional Examples from Duke Blue Pay:

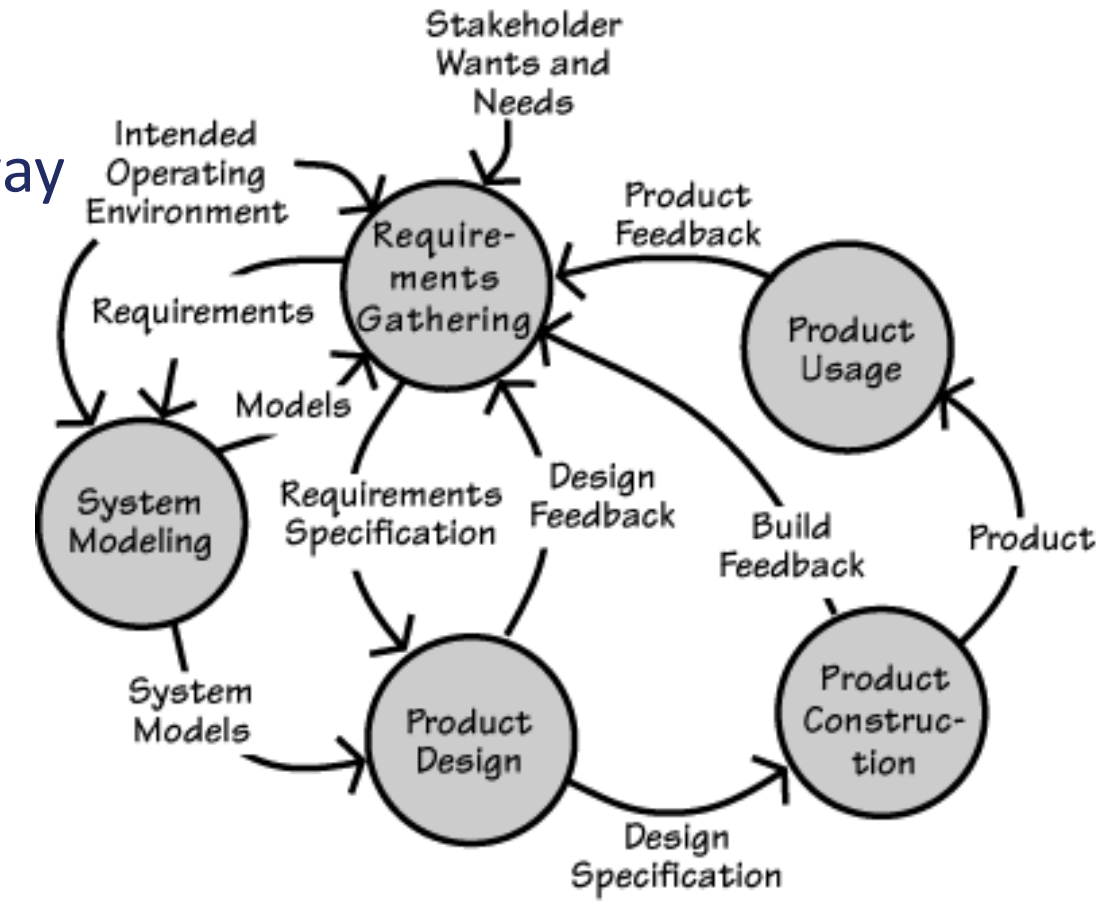
- The system shall allow a cardholder to add funds to their wallet from an external bank account.
- The system shall allow merchants to accept payments at POS using the Duke ID card (physical tap/swipe or barcode/QR).
- The system shall allow cardholders to send money to other cardholders.

- "The hardest single part of building a software system is deciding precisely what to build.
- No other part of the conceptual work is as difficult as establishing the detailed technical requirements
- No other part of the work so cripples the resulting system if done wrong.
- No other part is as difficult to rectify later."

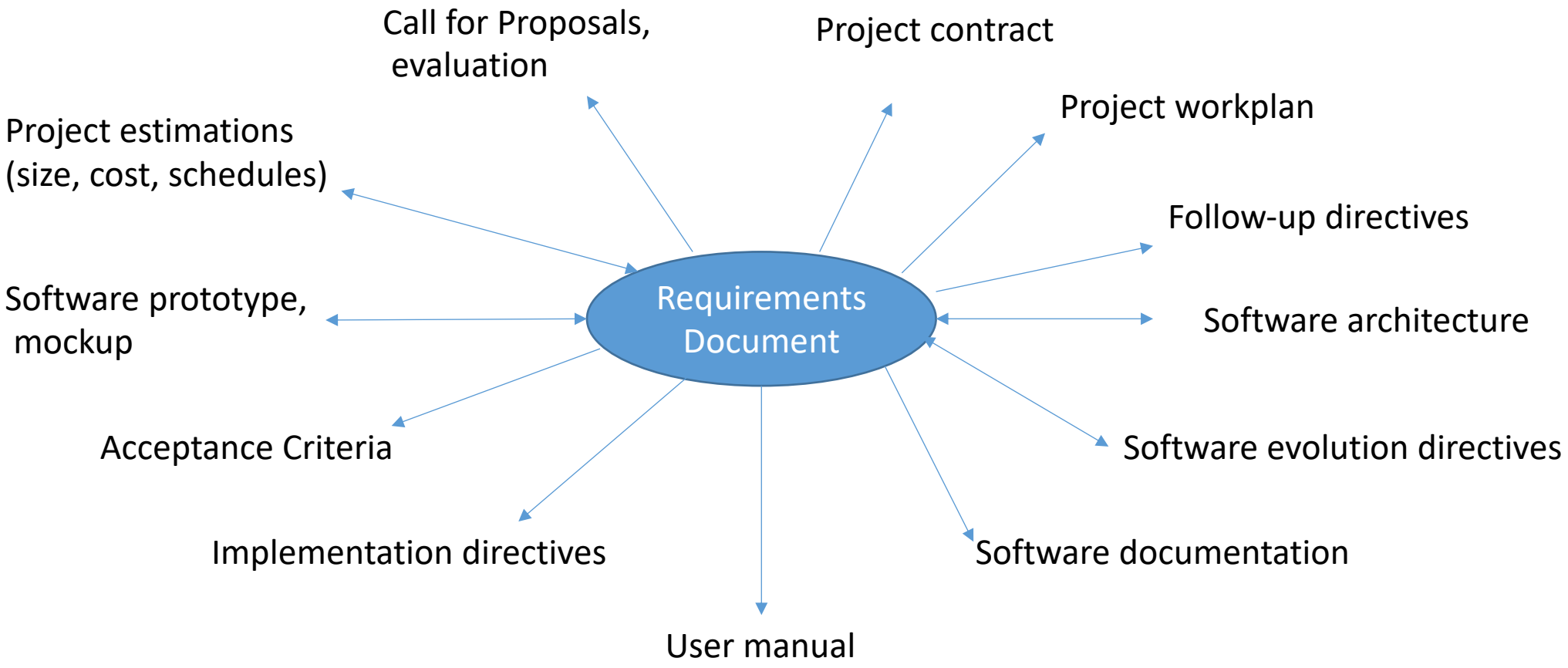
— Fred Brooks

Where do requirements come from?

- **Users:**
  who need the system to work a certain way
- **Business goals:**
  what the organization needs to achieve
- **Regulations:**
  legal and compliance constraints
- **Operational realities:**
  how the system will actually run
- **Failures:**
  lessons learned from what went wrong



Requirements Usage in Software Projects



Functional Requirements

What the system should do

- Input
- Output
- Interface
- Response to events

Nonfunctional Requirements

Constraints on behavior

Qualities the system should possess, e.g.:

- Availability
- Reliability
- Usability
- Performance
- Security

https://en.wikipedia.org/wiki/List_of_system_quality_attributes
https://en.wikipedia.org/wiki/Non-functional_requirement

Example requirements

- F: Users may transfer funds to another user.
- F: External vendors/business from Duke University must have the capability to accept payment a BluePay user either through presenting the Duke ID card or by presenting their mobile phone with the installed BluePay application.
- NF: Users must be authenticated to the system utilizing existing Duke authentication methods.
- NF: The system must be available 24 by 7.

Requirements Document

- Captures what stakeholders want and why
- Includes
  - Functional requirements
  - Non-functional requirements
  - Constraints and assumptions
- It should:
  - Be reviewed with stakeholders
  - Surface ambiguity and disagreement
  - Evolve as understanding improves

A requirements document is necessary, but not necessarily sufficient.

User Stories

Short, simple description of a feature told from the perspective of the person who desires the new capability

As a [type of user], I want to [perform some task] so that I can [achieve some goal].

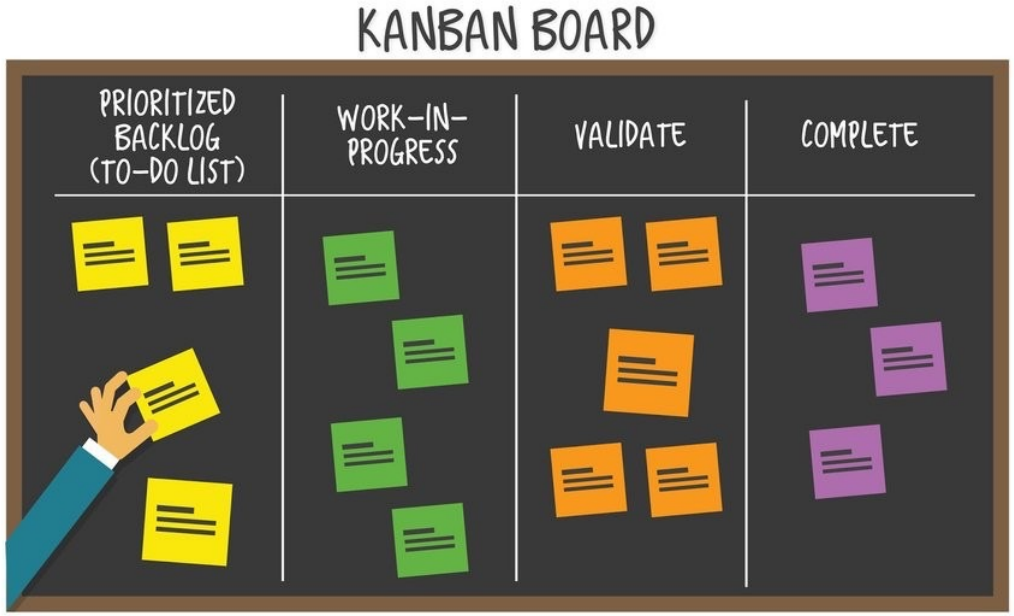
As a *student*, I want to *use my Duke ID card to pay for meals at Chipotle* so that I can *use funds from my flex plan rather than my personal savings*.

Often written on index cards or sticky notes -> becomes a discussion

User Story:
Card + Conversations + Acceptance Criteria

- **Card**: Document the story
- **Conversation**: Gain a common, improved understanding with the customer
- **Acceptance Criteria:** When is the story done
  - **Success –** what happens when everything works
  - **Failure –** what happens when something goes wrong
  - **Boundaries -** edge cases and limits

Acceptance criteria makes stories testable



Acceptance Criteria Example

As a *customer*
I want to *transfer money between my accounts*
So that I *can manage my finances*

Acceptance Criteria:

- Given sufficient funds, the transfer completes successfully
- Given insufficient funds, the transfer is rejected with a clear error
- Transfer amount must be greater than $0.01
- Source and destination accounts must be different
- The transaction appears in history within 1 second

Limits of User Stories

User stories are valuable, but they're not complete requirements

- Stories **don't replace analysis** — you still need to think deeply
- Stories **don't capture everything** — especially system-level constraints
- Stories **must be refined** into precise behavior

User stories are a starting point for conversation,
not an ending point for specification

Using Generative AI

- Provide context
- Provide sample outputs
- Detail output format (number and format)
- Assume a given role
- ”Least to Most” / Become more specific

Generative AI: Requirements

- Assist in drafting clear, concise requirements documentation.
- Extract key requirements from stakeholder interviews or existing documents.
- Provide templates and examples for user stories.
- Identify missing requirements or inconsistencies in existing documentation.
- Analyze requirements to identify critical features and dependencies.
- Ensure appropriate security controls are considered

Prompts

Suppose we have a system called "Duke BluePay," which is an extension of Duke's ID Card to allow students and employees of Duke University and the associated healthcare system to use their ID card just like other recent financial systems like PayPal, Stripe, and Venmo. Key features: 1. Web and mobile account access 2.Point of sale access 3. Social and vendor payments 4. Credit available

Create a detailed requirements document for this system.

What financial regulations should I be concerned about with Duke Blue Pay?

With Duke Blue Pay, list ten specific requirements to manage vendors with the system?

Using a business model canvas approach, list five value propositions for Duke Blue Pay.

---

what are the key risks associated with Duke Blue Pay and how likely will each occur?

Write a use story for Duke Blue Pay that allows a student to pay for a meal at a local restaurant. Include acceptance criteria

---

Design a screen to transfer funds from one student to that student's friends.

Ideation Prompt

Ask me one question at a time so we can develop a *thorough requirements specification* for this idea. Each question should build on my previous answers, and our end goal is to produce a *detailed specification* I can hand off to *a developer and give to stakeholders for review*. Let's do this iteratively and dig into every relevant detail. Remember, only one question at a time.

Here's the idea: I want to build a system called "Duke BluePay," which is an extension of Duke's ID Card to allow students and employees of Duke University and the associated healthcare system to use their ID card just like other recent financial systems like PayPal, Stripe, and Venmo. Key features: 1. Web and mobile account access 2.Point of sale access 3. Social and vendor payments 4. Credit available

https://chatgpt.com/share/e/69605c34-cc24-8013-9810-be3d34a196d4
(should be accessible through a Duke-based ChatGPT account)

Git and GitHub/GitLab

Not Just for code → engineering coordination mechanism

Version Control – Historical Viewpoint

- Backup and restore
- Synchronization
- Undo
- Track changes
- Sandbox
- Branch / merge

Git Is Not Just for Code

Version control is fundamentally about:

- A **history of decisions**
- A **coordination mechanism** for teams
- A **safety net** for experimentation

Version control is how teams remember why the system behaves the way it does.

What is git?

- Distributed version control system
- Created by Linus Torvalds
- Goals:
  - Speed
  - Non-linear development
  - Handle large projects efficiently

Warning - differs from traditional version control: SVN/CVS

Establish a local Repository:

1. Create a new one
   - Create a new directory and open it
   - `git init`
2. Create a working copy of another local
   - `git clone /path/to/repository`
3. Clone a remote repository
   - `git clone username@host:/path/to/repository`

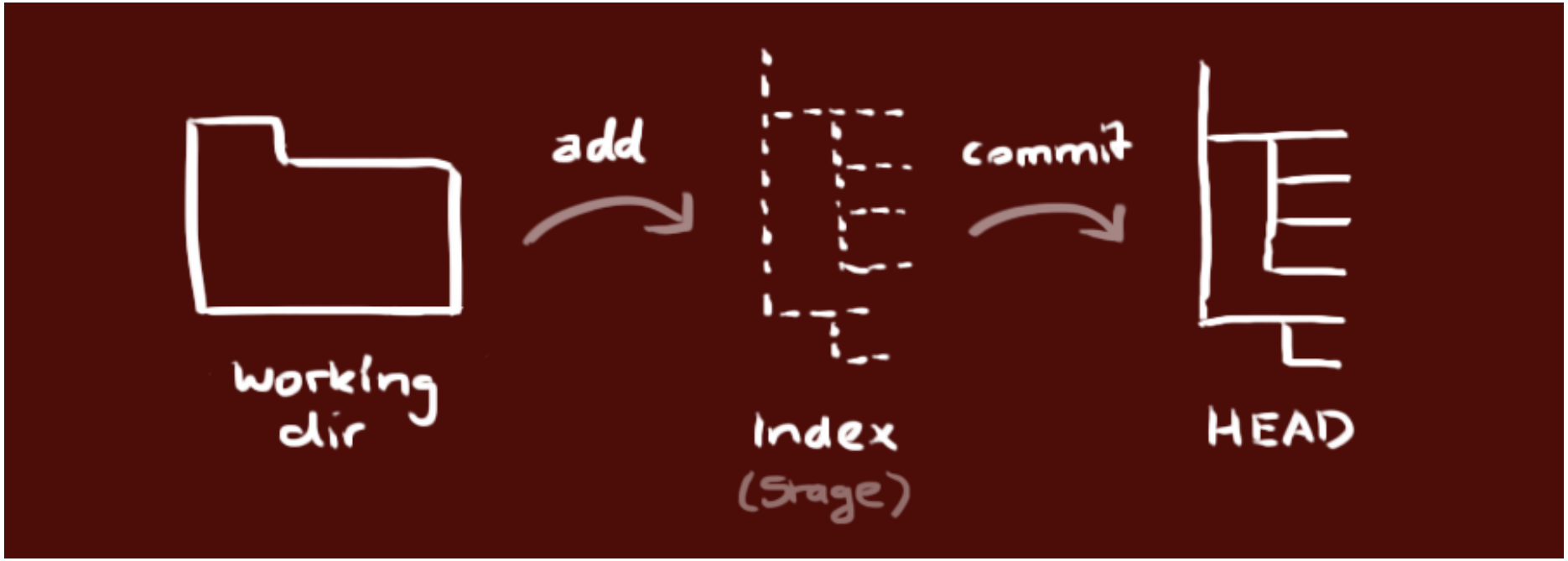
```
git clone https://gitlab.oit.duke.edu/netID/fintech-512-assignments.git
```

Workflow: your local repository

Working Directory: holds the actual files

Index: staging area

HEAD: points to the last commit made



Workflow: add & commit

Commit
saved snapshot of a set of changes to your project, with a message explaining what you changed and why

- View the repository status: `git status`
- Add changes to the Index:
  - `git add <filename>`
  - `git add *`

- To commit the changes:
  - `git commit –m "commit message"`

At this point, changes only exist in your local repository

- To push to the server:
  - `git push`
- To pull from server: (fork updated, …)
  - `git pull`

git resources

- GUI Clients
  - github.com: https://desktop.github.com/
  - SourceTree: https://www.sourcetreeapp.com/
  - VS Code Extension: https://www.gitkraken.com/gitlens

- Books / Guides
  - Pro Git: https://git-scm.com/book/en/v2
  - Version Control by Example: https://ericsink.com/vcbe/index.html

- Command "Cheat Sheets"
  - https://www.atlassian.com/dam/jcr:8132028b-024f-4b6b-953e-e68fcce0c5fa/atlassian-git-cheatsheet.pdf
  - https://about.gitlab.com/images/press/git-cheat-sheet.pdf

GitHub/GitLab Interfaces (Conceptual)

GitHub and GitLab provides collaboration tools on top of Git:

- **Repositories** — where code and history live
- **Issues** — requirements, bugs, discussions
- **Pull Requests** — review and shared understanding

Key insight:

- Issues = requirements / bugs / discussions
- PRs = review and shared understanding before merging

Duke's GitLab Server: gitlab.oit.duke.edu

Search or go to... /

0 1 0

Project

F Fintech 512 2026 Assignments

Pinned

Issues 0

Merge requests 0

General

Members

Manage

Plan

Code

Build

Secure

Deploy

Operate

Monitor

Analyze

Settings

FINTECH 512 / Fintech 512 2026 Assignments

F Fintech 512 2026 Assignments

Star 0 Fork 6

main fintech-512-2026-assignments Find file Code

timings update
John Slankas authored 1 day ago

beeb7225 History

Name	Last commit	Last update
images	update fork text	2 days ago
week1	timings update	1 day ago
week2/KWIC	initial work	2 days ago
.gitignore	initial work	2 days ago
PSPScript_version2.docx	initial work	2 days ago
README.md	update fork text	2 days ago
requirements.txt	initial work	2 days ago

README.md

Fintech 512 2026 Assignments

This project provides the shell for the weekly assignments as well as external code used in the class.

Each student must create a private fork of this repository and provide their instructor and the course TA's with access.

Project information

6 Commits

1 Branch

0 Tags

760 KiB Project Storage

README

Add LICENSE

Add CHANGELOG

Add CONTRIBUTING

Enable Auto DevOps

Add Kubernetes cluster

Set up CI/CD

Add Wiki

Configure Integrations

Created on
January 06, 2026

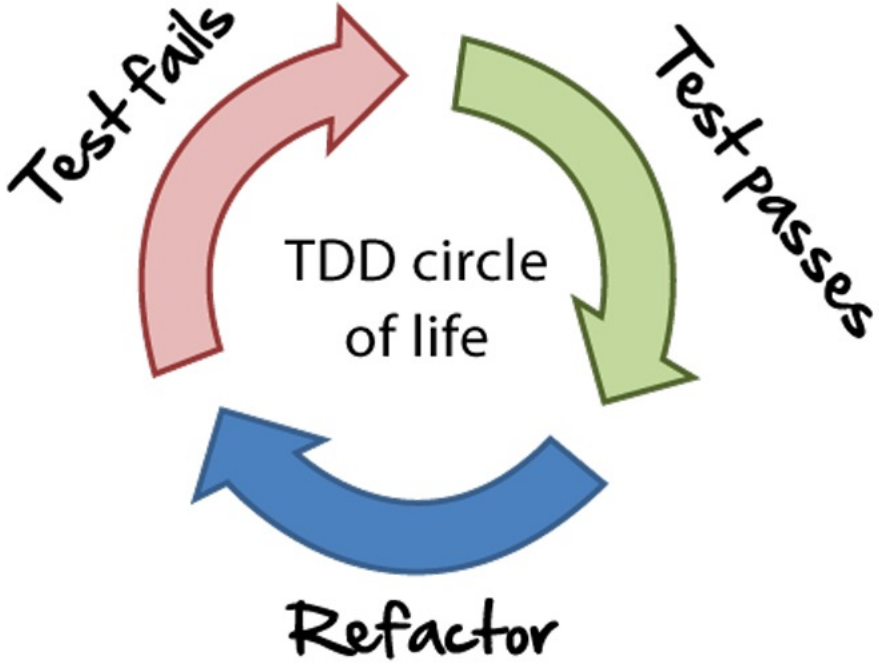
Duke

Test-Driven Development

Decide what ‘correct’ means before writing the code.

Test Driven Development (TDD)

1. Decide what the test will do
2. Write the test code
3. Watch the test fail
4. Write test logic as simple as possible
5. Pass the test
6. Refactor, removing any duplicates
7. Repeat until complete



TDD as Design, Not Testing

This is the conceptual heart of test-driven development:

- **Tests force precision** — vague requirements become concrete
- **Tests encode decisions** — they document intended behavior
- **Tests make requirements executable** — the system proves it works

TDD is the most precise requirements language we have.

Why TDD Matters in Finance

Test-driven development directly addresses critical concerns:

- **Correctness** — financial calculations must be exact
- **Idempotency** — retrying the same operation should be safe
- **Edge cases** — what happens at boundaries?
- **Regression prevention** — new changes don't break old behavior

Critical scenarios:

- Duplicate payment detection
- Retry behavior after network failure
- Rounding in compound interest calculations

What TDD Does NOT Do

Important to set realistic expectations:

- TDD **doesn't replace thinking** — you still need good requirements
- TDD **doesn't guarantee correctness** — tests can be wrong too
- TDD **doesn't eliminate design** — you still need architecture and design

TDD is a discipline that helps us think more clearly,
not a substitute for thinking

Unit Testing != TDD

TDD is a design methodology,

- it creates unit tests
- forces you to make certain design decisions

You can do unit testing without TDD, but you can’t do TDD without unit tests
When you do traditional unit testing, you write test after you wrote your code.
Test driven development approach writes unit test before writing code

- Code is fully tested code upfront.

pytest

- Alternate test framework for Python
- Tests are plain Python functions; no need for classes

```
def test_string_concatenation():
    assert "Hello" + " World" == "Hello World!"

def test_list_length():
    my_list = [1, 2, 3]
    assert len(my_list) == 3
```

```
pytest test_built.py
========================================================================== test session starts ==========================
platform darwin -- Python 3.12.0, pytest-7.4.4, pluggy-1.3.0
rootdir: /Users/jbslanka/Documents/GitHub/fintech-510-instructor-resources
plugins: anyio-4.0.0
collected 2 items
test_built.py F. [100%]
================================================================================ FAILURES ===================================
_____________________________________________________________________ test_string_concatenation _____________________________
def test_string_concatenation():
> assert "Hello" + " World" == "Hello World!"
E AssertionError: assert 'Hello World' == 'Hello World!'
E - Hello World!
E ? -
E + Hello World
test_built.py:2: AssertionError
========================================================================= short test summary info ===================
FAILED test_built.py::test_string_concatenation - AssertionError: assert 'Hello World' == 'Hello World!'
=================================================================== 1 failed, 1 passed in 0.03s ==================
```

Duke

Good testing …

- Automated
- Repeatable
- Continuous

- Isolated

What to test?

Test your code

… not your dependencies

Generally, assume widely adopted libraries, frameworks, operating systems, etc. have been tested and work as advertised

TDD Example

Implement a bank account that tracks the account number, balance and allows credits/debits

Directory Structure

```
banking_system/
    app/
        bank_account.py
    test/
        test_bank_account.py
```

Write Initial Test Case

```
def test_banking_credit_method_returns_correct_result():
        bank = BankAccount()
        final_bal = bank.credit(1000)
        self.assertEqual(1000, final_bal)
```

Run the Test Case

```
Line x: bank = BankAccount()
NameError name ‘BankAccount’ is not defined
```

Implement (initial) BankAccount class

```
class BankAccount:
    def __init__(self):
        self.balance = 0

    def credit(self, amount):
        pass
```

Update test case

```
from app.bank_account import BankAccount

def test_banking_credit_method_returns_correct_result():
     bank = BankAccount()
     final_bal = bank.credit(1000)
     self.assertEqual(1000, final_bal)
```

Run the test again

```
AssertionError: 1000 != None
Ran 1 test in 0.002 s FAILED(failures = 1)
```

Update the BankAccount Class to pass test

```
class BankAccount:
    def __init__(self):
        self.balance = 0


    def credit(self, amount):
        self.balance += amount
        return self.balance
```

Run the test again

```
Testing started at ...
Ran 1 test in 0.001 s
OK
```

Now add more to the test case (or a new one)

- Validate the amount is a number
- Debit
- Getting the balance

Unit testing: Arrange, Act, Assert (AAA) Pattern

```
from app.bank_account import BankAccount

def test_banking_credit_method_returns_correct_result():
    # arrange (setup variables/environment)
    bank = BankAccount()
    # act (perform some operation(s)
    final_bal = bank.credit(1000)
    # assert (validate state after actions)
    self.assertEqual(1000, final_bal)
    # ”tear down” – perform any additional cleanup
```

Pulling It Together

Today we've connected several key ideas:

- **Requirements define behavior** — what the system must do
- **Stories capture intent** — why the behavior matters
- **Git records decisions** — how and why things changed
- **Tests enforce expectations** — proof that requirements are met

These aren't separate tools — they're different views of the same thing: **engineering decisions**

We now know why requirements matter, why stories aren’t enough, and why tests enforce decisions. Next time, we’ll focus on the missing middle: precisely specifying system behavior.